WeCloudData

LLM Fine-tuning

Introduction

LLM Fine-tuning

Introduction

Building and Tuning LLMs

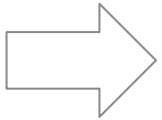
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

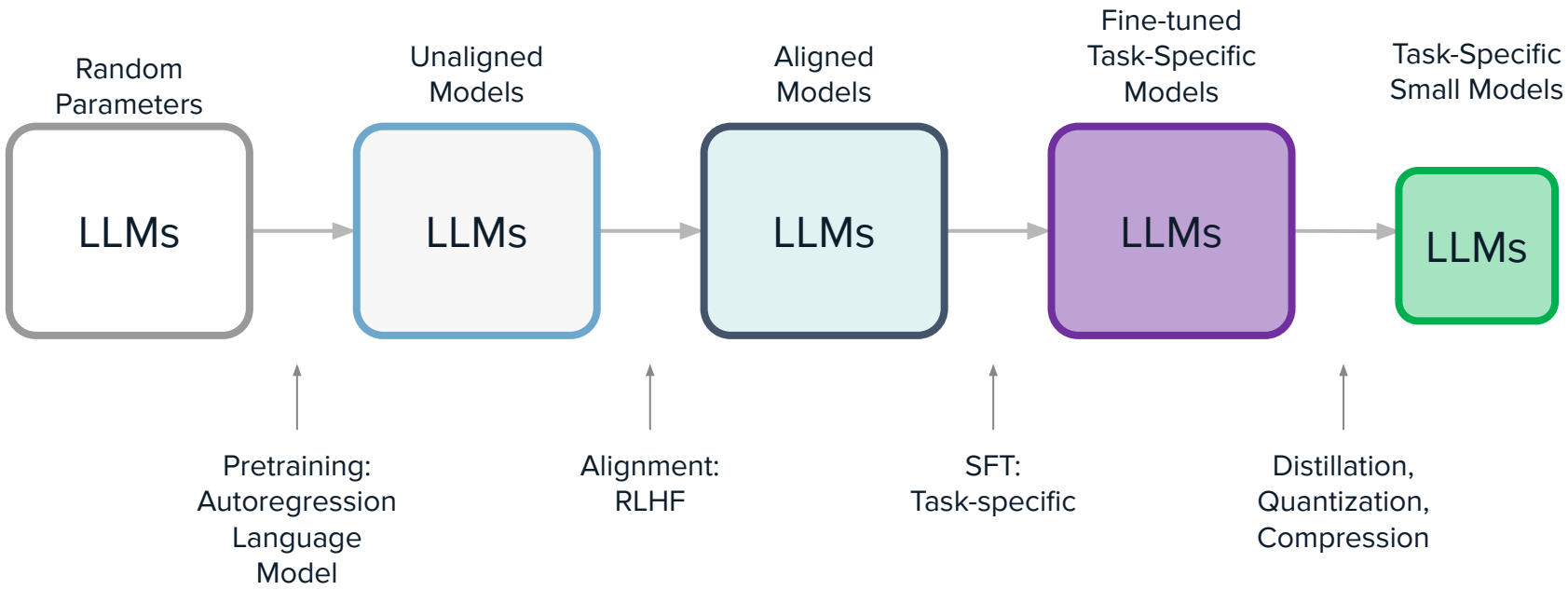
Fine-tuning

Parameter Efficient Fine-tuning

Agenda.

The LLM Lifecycle

Fine-tuning Introduction



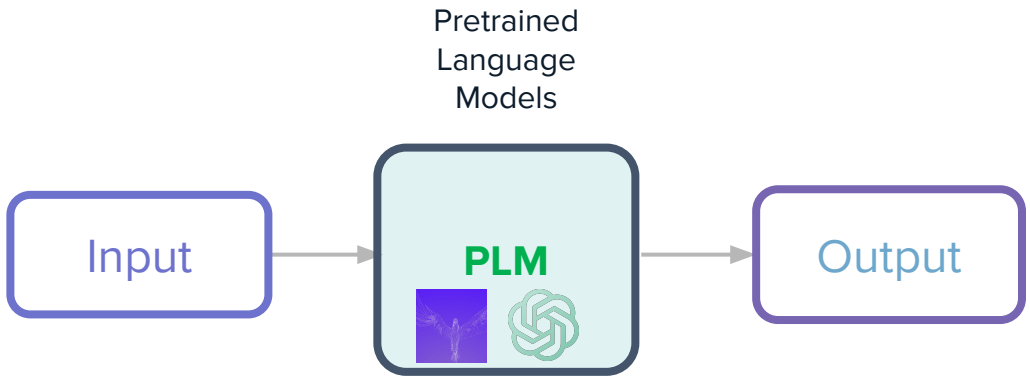
Using LLMs for Inference

Fine-tuning Introduction

In the previous module, we learned

- Attention mechanism
- Transformer architecture
- Different LLMs

When it comes to use cases, we mainly used pretrained models for inference tasks.

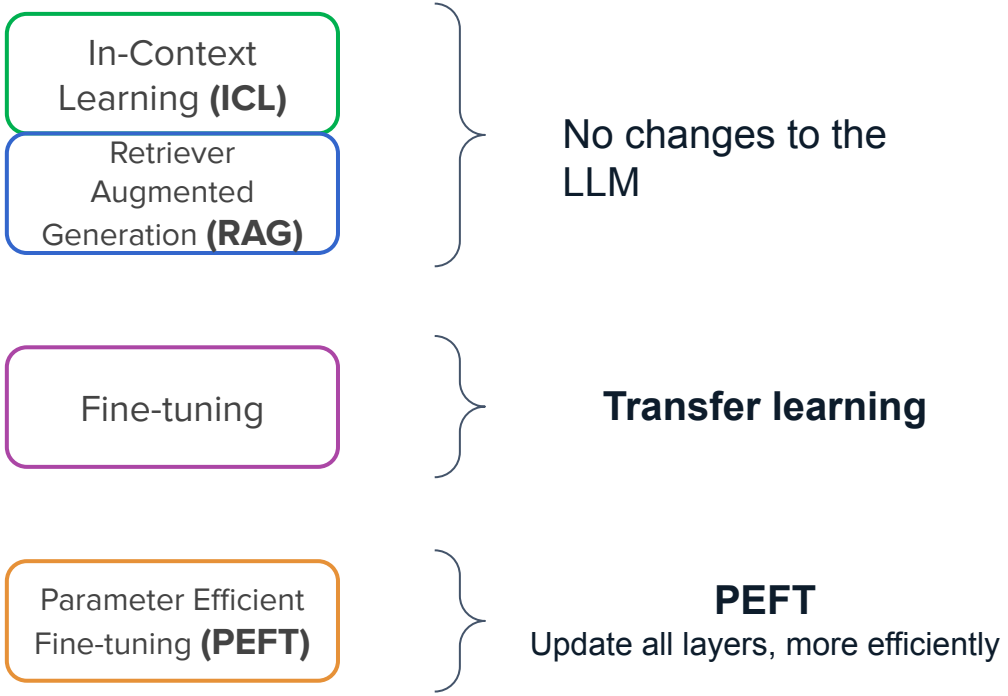


How do we ask LLMs to perform specific tasks?

Fine-tuning Introduction

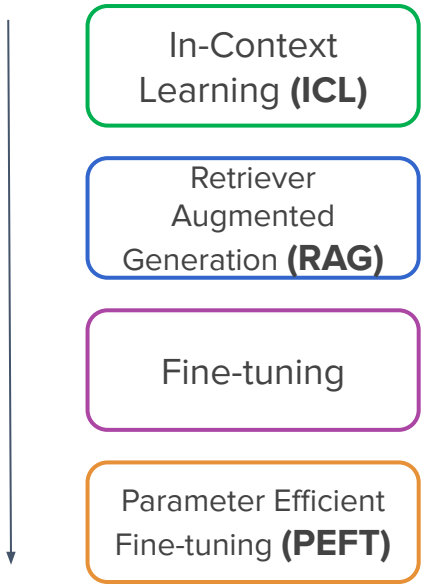
However, in practice we always want to build task-specific solutions.

- We want our LLMs to be tailored to different domains
- We want our LLMs to be trained on private data
- We want our LLMs to not only follow instructions but also perform different tasks



How do we approach this course?

Fine-tuning Introduction



- In this lecture, we will give a high-level overview of different methods and focus on ICL
- In the next lecture we will learn Indexing method such as FAISS and RAG
- In the later lectures, we will focus more on fine-tuning and PEFT
- In the Module 3.2 - NLP course, we will learn pre-training techniques

LLM Fine-tuning

Pretraining

Building and Tuning LLMs

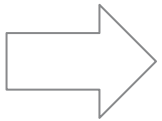
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

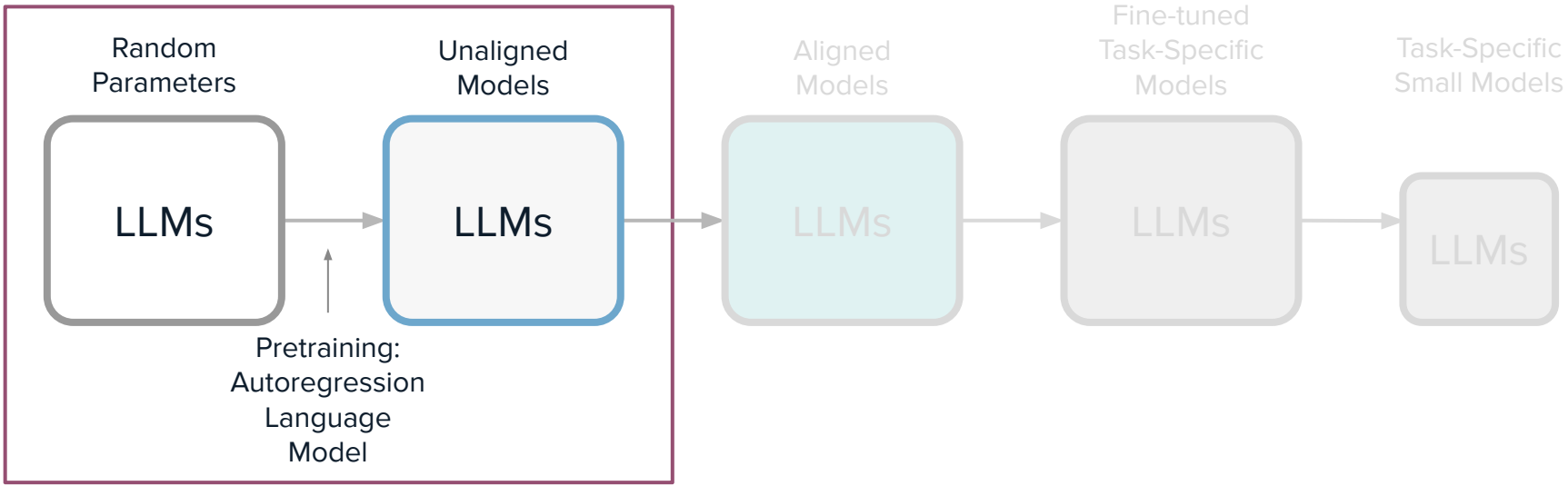
Fine-tuning

Parameter Efficient Fine-tuning

Agenda.

Pre-training

Pre-Training Introduction



Language models are pre-trained on massive text data to predict the next word (ARM). The model can blabber words but may not always give what we want.

Pre-training Phase

WeCloudData

Pre-trained LLMs are next-token predictors

Pre-Training Introduction

The language model of a series of transformer layers. Each receives a set of word embeddings and outputs a set of processed word embeddings. At the final layer, the last embedding is mapped via a linear transformation and softmax function to a probability distribution over possible next tokens.

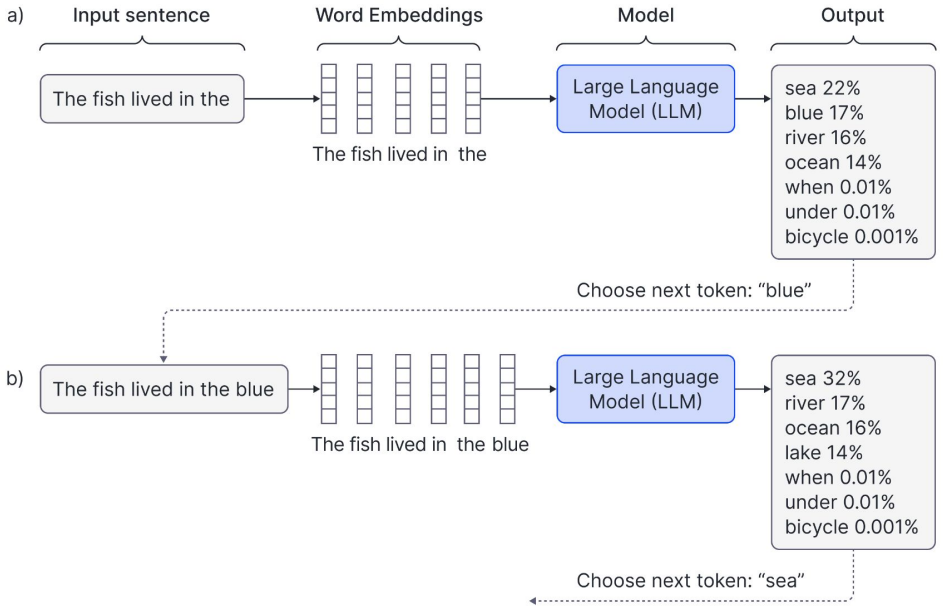


Image source: Training and fine-tuning large language models (blog)

Example: Transformer Decoder Network

Pre-Training Introduction

The example here is a transformer decoder network, of which the goal is to predict the next word in a partially complete input string.

- The input string is divided into tokens, each of which represents a word or a partial word.
- Each token is mapped to a corresponding fixed-length *embedding*.
- The sequence of embeddings that represent the sentence is fed into the decoder model, which predicts a probability distribution over possible next tokens in the sequence

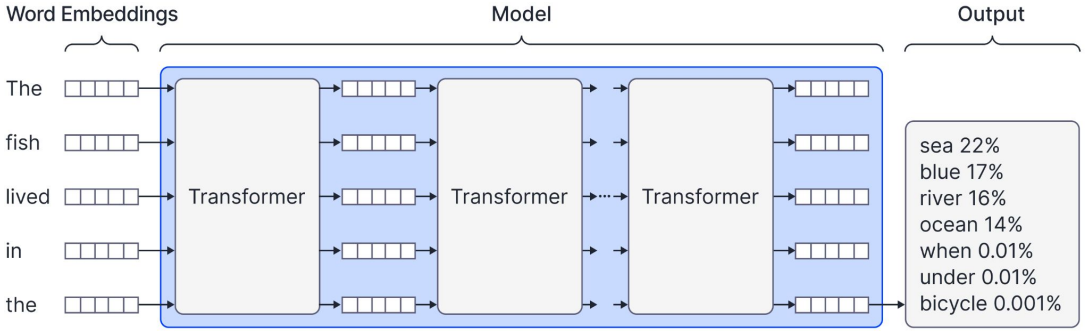


Image source: Training and fine-tuning large language models (blog)

Inside Each Transformer Decoder Block

Pre-Training Introduction

Each individual transformer layer consists of a masked self-attention layer (in which the embeddings interact with one another) and a set of parallel fully connected networks (in which the embeddings a processed individually).

In practice, other components, such as residual connections and layer normalization, are also included to make the model easier to train (not shown here).

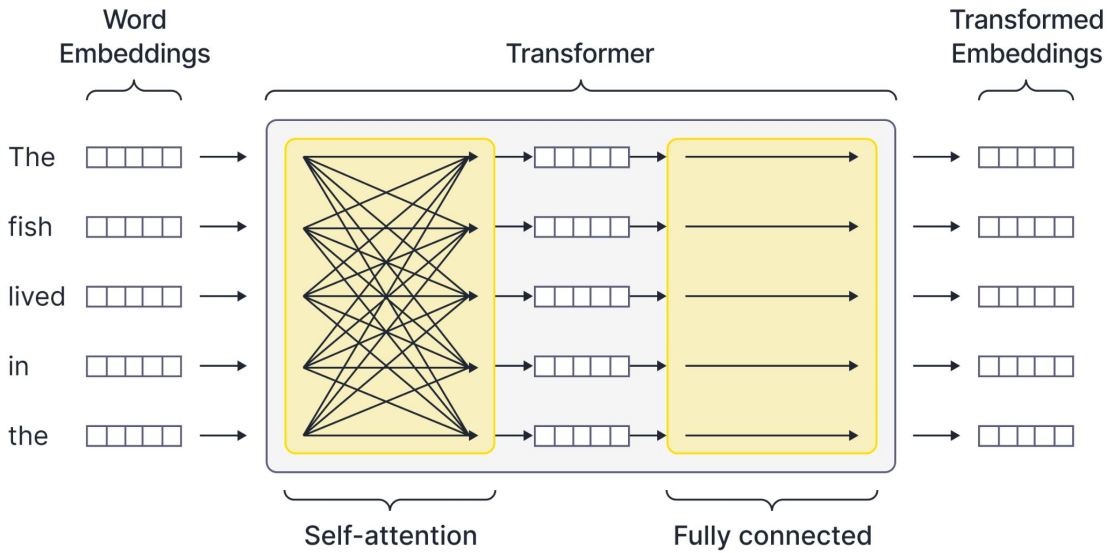


Image source: Training and fine-tuning large language models (blog)

Masked Self-Attention

Pre-Training Introduction

To make training more efficient, the model is modified so that every output embedding (i.e., not just the last embedding) predicts the subsequent token. The first input token becomes a special *<start>* token, and the first output must predict the first word. The second input is the first word, and the second output must predict the second word, and so on. After training, when we generate new text with the model, we use only the final output to predict the next word.

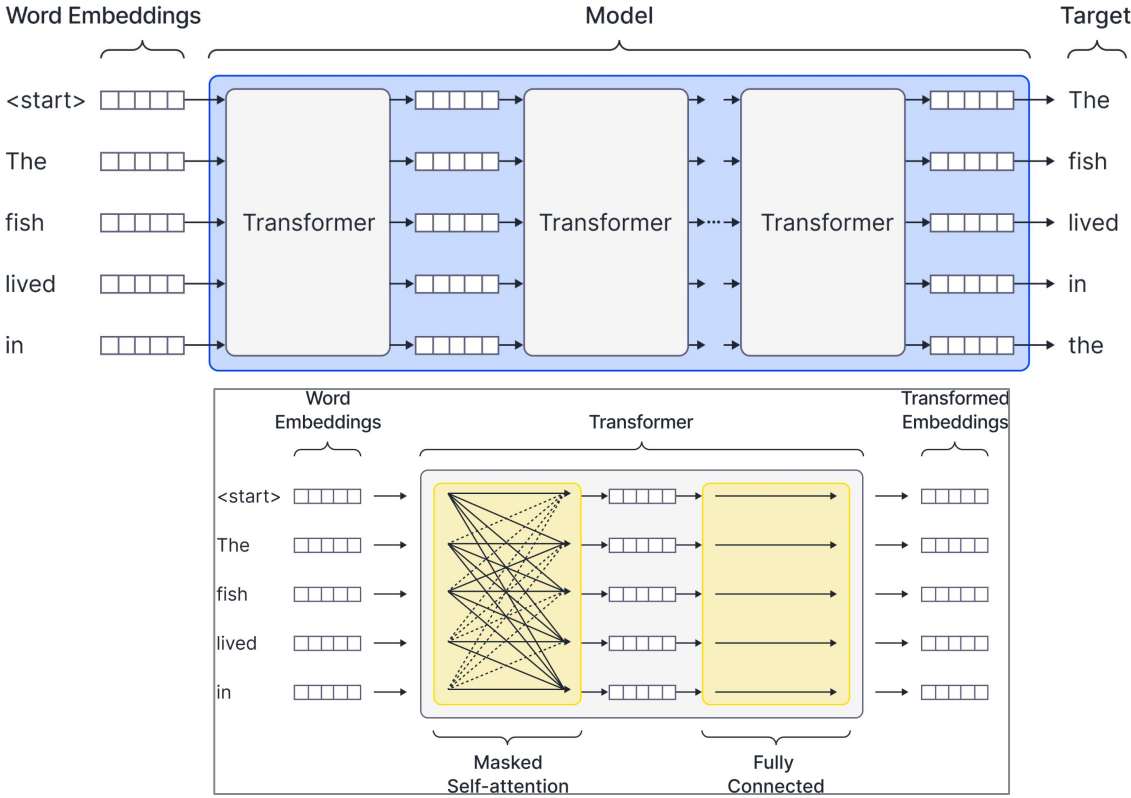


Image source: Training and fine-tuning large language models (blog)

The Pre-training Process

Pre-Training Introduction

When pre-training a LLM

- Model has zero knowledge about the world at the start
- When the training process starts, the model starts to learn how to predict the next token
- It requires massive corpus of text data that are often scraped from the internet
- The training data for pre-training stage is not label since it’s self-supervised
- Training is a very expensive and time consuming effort
- After the training, the LLM starts to have “knowledge” about the world and can blabber sentences.

The “secret” sauce of during pre-training

Pre-Training Introduction

The performance of a pre-trained LLM depends on:

- Well curated and prepared pre-training datasets
  - Note that data used for actual training is usually not made publicly, though OSS LLMs share information about the raw data sources
- GPU clusters for model training
  - It requires thousands of GPU cards to pre-train LLMs
- $$$

We will learn more about pre-training in the M3.2 NLP course, and computation distribution in M3.4.

Limitation of pretrained base model

Pre-Training Introduction

The pre-trained LM contains the probability distribution of given a context that predict the next possible token. When the context is a question, or related to some domain specific text, the pre-trained LM may show a sign of “understanding” the instruction and “response” to it.

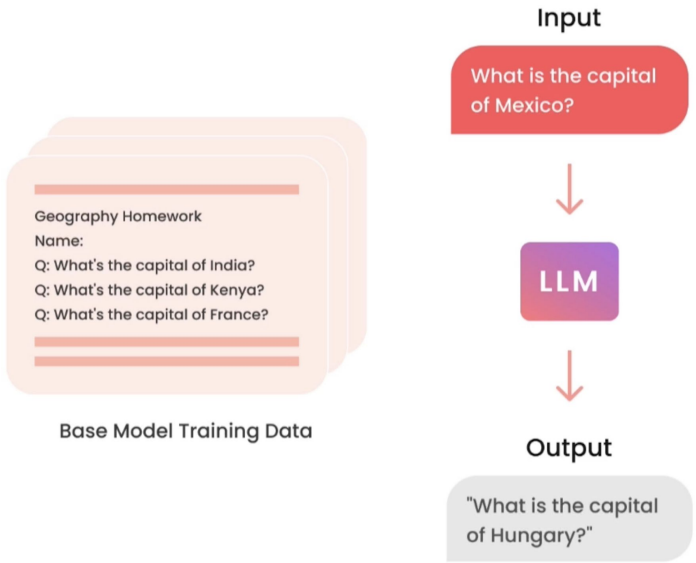


Image adapted from Lamini’s deeplearning.ai fine-tuning lecture

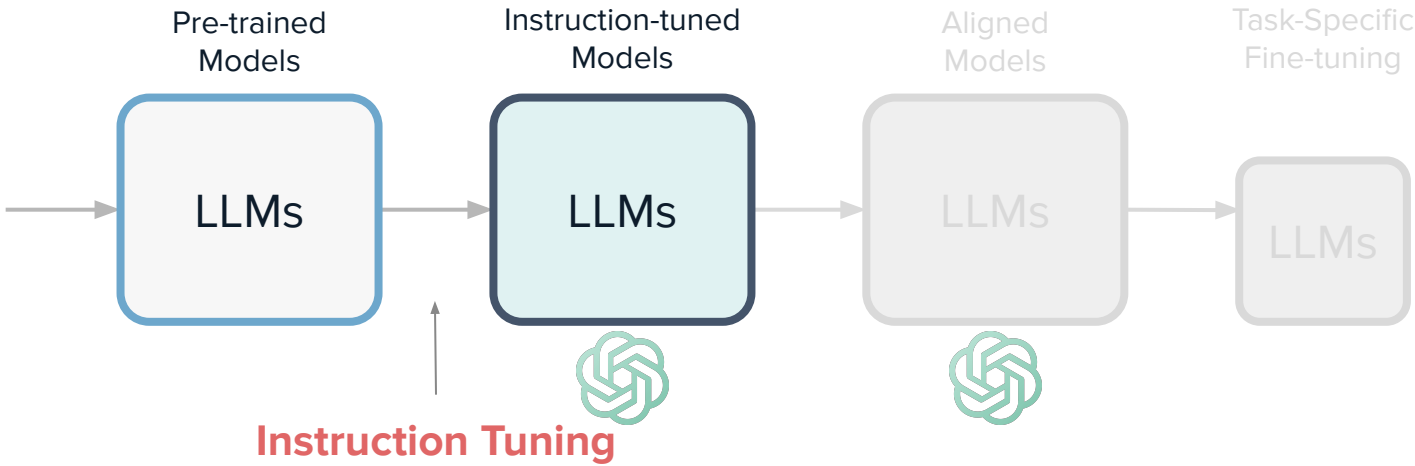
Instruction Tuning

Pre-Training Introduction

So how do we make a pre-trained LLM follow our instructions?

Instruction Fine-tuning

Fine-tuning Introduction



WeCloudData

LLM Fine-tuning

Fine-tuning Intro

Building and Tuning LLMs

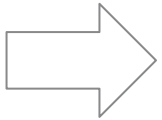
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

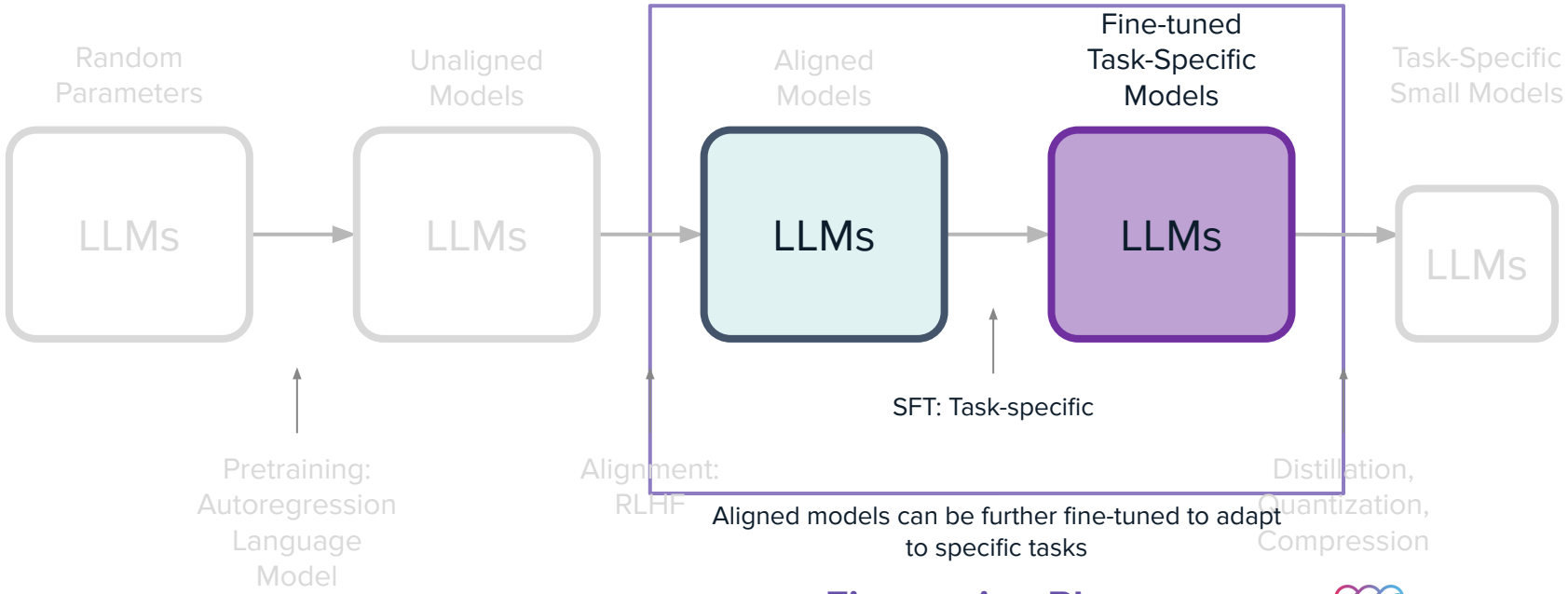
Fine-tuning

Parameter Efficient Fine-tuning

Agenda.

The Need for Fine-tuned LLMs

Fine-tuning Introduction



Fine-tuning Phase

WeCloudData

Purpose of Fine-tuning

Fine-tuning Introduction

Researchers fine-tune LLMs for different purposes

- Behavior change
  - Learning to respond more consistently
  - Learning to focus, e.g. moderation
  - Teasing out capability, e.g. better at conversation
- Gain new knowledge
  - Increasing knowledge of new specific concepts
  - Correcting old incorrect information
- Both

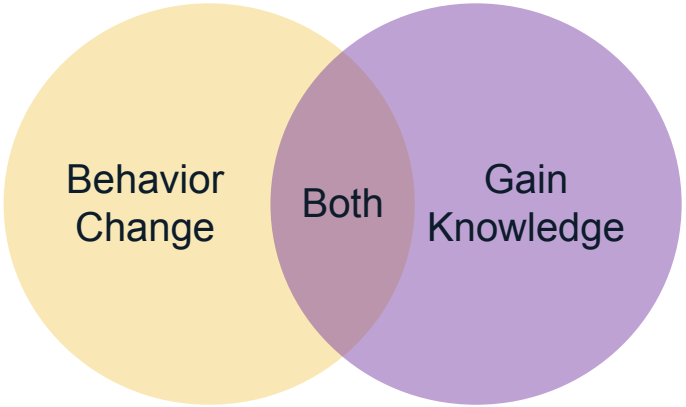


Image adapted from Lamini's deeplearning.ai fine-tuning lecture

Fine-tuning Tasks

Fine-tuning Introduction

- Fine-tuning usually refers to training further
  - Can also be self-supervised unlabeled data
  - Can be “labeled” data you curated
  - Much less data is needed usually
- Fine-tuning is also task-specific
  - Extraction related tasks
    - Text in, less text out
    - Keywords, topics, reasoning, routing, agents, etc.
  - Expansion
    - Text in, more text out
    - Chat, write emails, write code

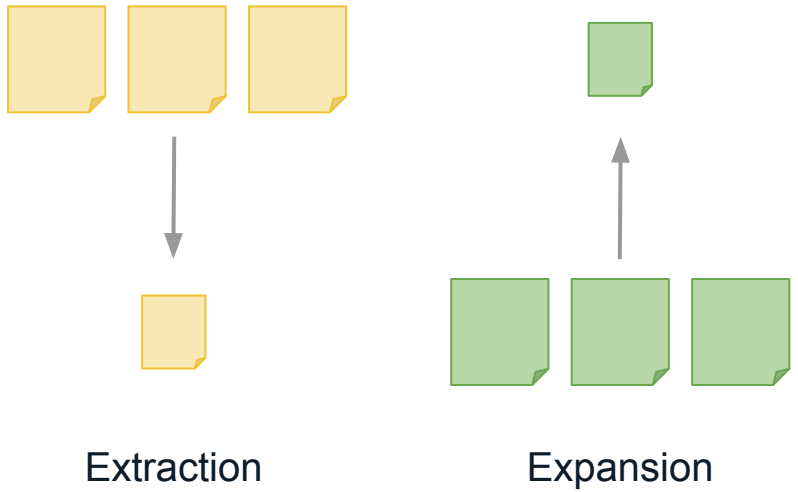


Image adapted from Lamini’s deeplearning.ai fine-tuning lecture

What does fine-tuning do?

Fine-tuning Introduction

- Allows more data into the model than prompt context
- Model learns from the data and trains additional parameters
- More consistent output
- Reduces hallucination
- Customize the model to a specific purpose

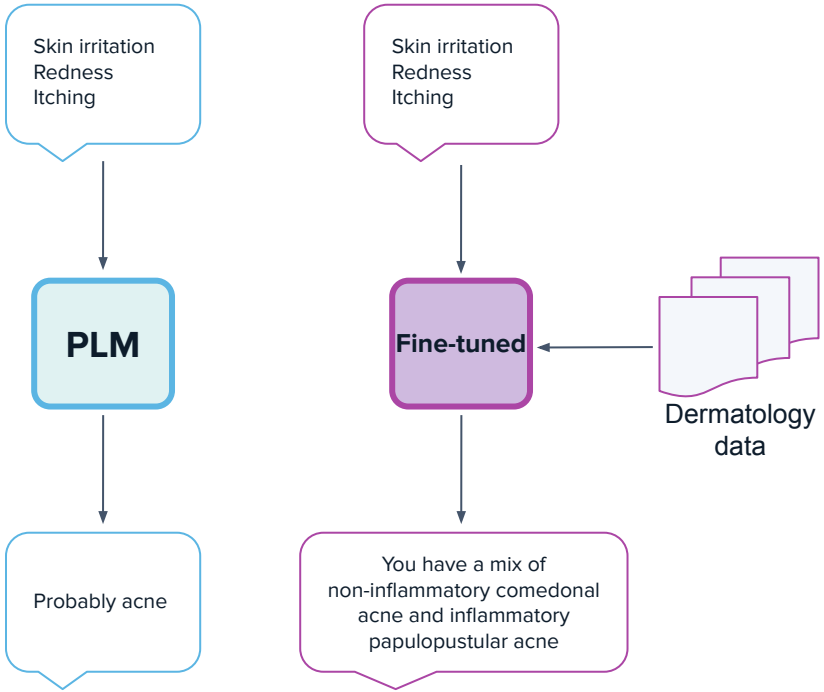


Image adapted from Lamini's deeplearning.ai fine-tuning lecture

What does fine-tuning do?

Fine-tuning Introduction

- Allows more data into the model than prompt context
- Model learns from the data and trains additional parameters
- More consistent output
- Reduces hallucination
- Customize the model to a specific purpose

Image adapted from Lamini's deeplearning.ai fine-tuning lecture

Pre-training LLM is not for everyone

Fine-tuning Introduction

Pre-training	Fine-tuning	ICL
1000x H100 (80G) GPU	**10x** H100 (80G) GPU	No GPU training required
10000 hours	**100** hours	Seconds
10TB+ data	**5-10k rows**	Few-shots
$10 million +	$5k~$10k	Inference cost

An analogy

Fine-tuning Introduction

PLM (Pre-trained)	Fine-tuning	Prompting
Middle/High School	**College**	**Job**
Spent 10 years learning the fundamentals • Math • Writing • Geography • Chemistry • etc.	Students choose different majors and become specialized • Accounting • Marketing • CS • EE • Etc.	Rarely attend classes any more • Learn on the job!

Prompting vs Fine-tuning

Fine-tuning Introduction

	Prompting	Finetuning
Pros	• No data to get started • Smaller upfront cost • No technical knowledge needed • Connect data through retrieval (RAG)	• Nearly unlimited data fits • Learn new information • Correct incorrect information • Less cost afterwards if smaller model • Use RAG too
Cons	• Much less data fits • Forgets data • Hallucinations • RAG misses, or gets incorrect data	• More high-quality data • Upfront compute cost • Needs some technical knowledge, esp. data

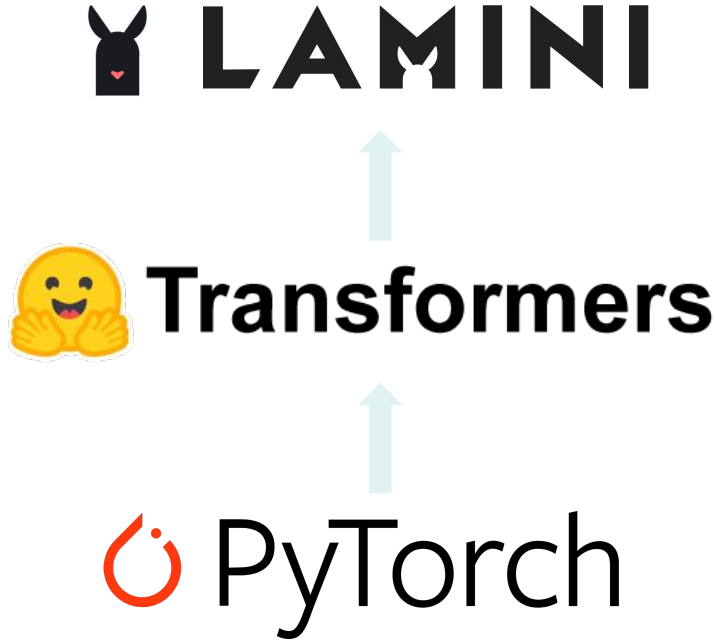
Benefits of Fine-tuning

Fine-tuning Introduction

Performance	• stop hallucinations • increase consistency • reduce unwanted info
Privacy	• on-prem or VPC • prevent leakage • no breaches
Cost	• lower cost per request • increased transparency • greater control
Reliability	• control uptime • lower latency • moderation

What tools are used for fine-tuning?

Fine-tuning Introduction



Fine-tuning Steps

Fine-tuning Introduction

Identify tasks by prompt-engineering a large LLM

Collect Instruction-Response Pairs

Concatenate pairs

Tokenize: pad, truncate

Fine-tune: Training

Fine-tuning is still costly. Always try prompting the LLM first and see how well it performs and identify the needs for fine-tuning.

Fine-tuning Steps

Fine-tuning Introduction

Identify tasks by prompt-engineering a large LLM

Collect Instruction-Response Pairs

Concatenate pairs

Tokenize: pad, truncate

Fine-tune: Training

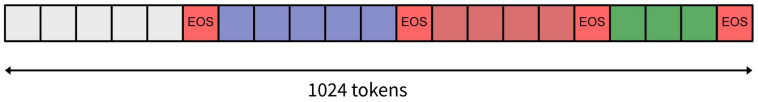
```
{
    "instruction": "Write a limerick about a
                    pelican.”,
    "input": "",
    "output": "There once was a pelican so fine,
              \nHis beak was as colorful as
              sunshine,\nHe would fish all day,\nIn
              a very unique way,\nThis pelican was
              truly divine!\n\n\n"
},


{
    "instruction": "Identify the odd one out from
                    the group.",
    "input": "Carrot, Apple, Banana, Grape",
    "output": "Carrot\n\n"
},
```

Fine-tuning Steps

Fine-tuning Introduction

- Identify tasks by prompt-engineering a large LLM
- Collect Instruction-Response Pairs
- **Concatenate pairs**
- Tokenize: pad, truncate
- Fine-tune: Training

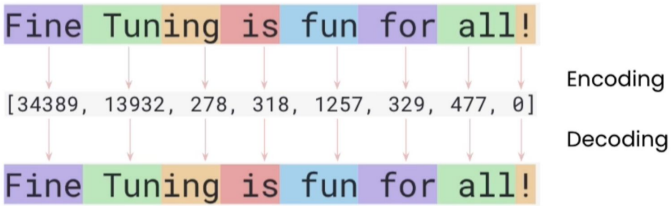


To make training more efficient and use the longer context of these LLMs we'll do something called "packing". We will combine multiple examples to fill the model's memory and make training more efficient instead of feeding examples individually. This way we avoid doing a lot of padding and dealing with different lengths.

Fine-tuning Steps

Fine-tuning Introduction

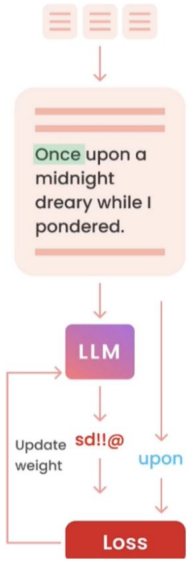
- Identify tasks by prompt-engineering a large LLM
- Collect Instruction-Response Pairs
- Concatenate pairs
- **Tokenize: pad, truncate**
- Fine-tune: Training



Fine-tuning Steps

Fine-tuning Introduction

- Identify tasks by prompt-engineering a large LLM
- Collect Instruction-Response Pairs
- Concatenate pairs
- Tokenize: pad, truncate
- **Fine-tune**: Training



Training is just like any other neural networks.

- Add training data
- Calculate loss
- Backprop through model
- Update weights

Hyperparameters

- Learning rates
- Learning rate scheduler
- Optimizer hyperparameters

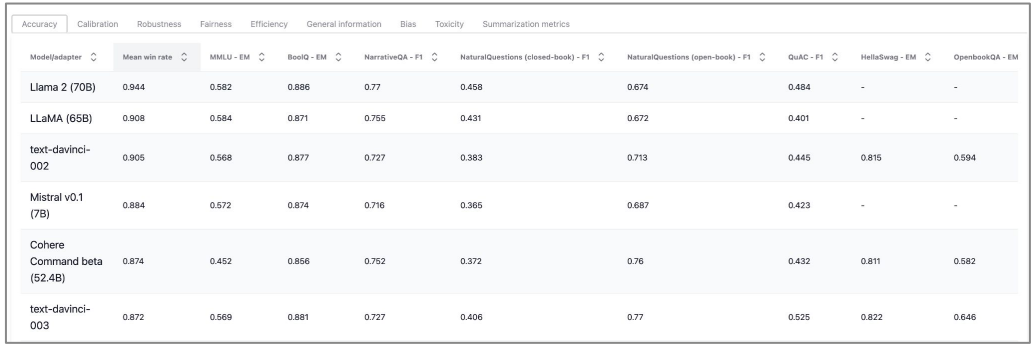
Image adapted from Lamini's deeplearning.ai fine-tuning lecture

Evaluation

Fine-tuning Introduction

Evaluating generative models is a critical step, and yet notoriously difficult.

- Human expert evaluation is more reliable but less scalable
- Good test data is crucial
  - High-quality
  - Accurate
  - Generalized
  - Not seen in training data
- Elo comparisons also very popular



Accuracy | Calibration | Robustness | Fairness | Efficiency | General information | Bias | Toxicity | Summarization metrics

Model/adapter	Mean win rate	MMLU - EM	BoolQ - EM	NarrativeQA - F1	NaturalQuestions (closed-book) - F1	NaturalQuestions (open-book) - F1	QuAC - F1	HellaSwag - EM	OpenbookQA - EM
Llama 2 (70B)	0.944	0.582	0.886	0.77	0.458	0.674	0.484	-	-
LLaMA (65B)	0.908	0.584	0.871	0.755	0.431	0.672	0.401	-	-
text-davinci-002	0.905	0.568	0.877	0.727	0.383	0.713	0.445	0.815	0.594
Mistral v0.1 (7B)	0.884	0.572	0.874	0.716	0.365	0.687	0.423	-	-
Cohere Command beta (52.4B)	0.874	0.452	0.856	0.752	0.372	0.76	0.432	0.811	0.582
text-davinci-003	0.872	0.569	0.881	0.727	0.406	0.77	0.525	0.822	0.646

Leaderboard Example

We have a session dedicated to evaluation

Fine-tuning GPU Requirements

Fine-tuning Introduction

Fine-tuning requires GPU resources. We will cover GPUs in *M3.4 Computation Distribution* course in more detail. In this course, we will mainly use V100/T4 types provided via Colab Pro, and AWS EC2

AWS Instance	GPUs	GPU Memory	Max inference size (# of params)	Max training size (# of tokens)
p3.2xlarge	1 V100	16GB	7B	1B
p3.8xlarge	4 V100	64GB	7B	1B
p3.16xlarge	8 V100	128GB	7B	1B
p3dn.24xlarge	8 V100	256GB	14B	2B
p4d.24xlarge	8 A100	320GB HBM2	18B	2.5B
p4de.24xlarge	8 A100	640GB HBM2e	32B	5B

Different Types of Fine-tunings

Fine-tuning Introduction

Topics we will cover

Pre-training	M2.4 ❌	M3.2 (NLP) ✅
Instruction Tuning	M2.4 ✅	
RLHF	M2.4 ❌	
In-Context Learning	M2.4 ✅	
Indexing (Prompt & RAG)	M2.4 ✅	M3.2 (NLP) ✅
Adaptors	M2.4 ✅	
PEFT	M2.4 ✅	

LLM Fine-tuning

Instruction Tuning

Building and Tuning LLMs

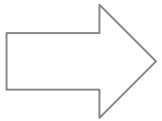
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

Fine-tuning

Parameter Efficient Fine-tuning

Agenda.

Some examples of fine-tuned LLMs

Instruction Tuning

Many pre-trained LLMs are decoder models that predict next tokens. They compressed massive amounts of “knowledge” into the model weights. However, when it needs to overcome a few challenges before considered to be useful in many scenarios

- It needs to be finetuned to follow specific instructions
- It needs to be aligned to give less biased and toxic answers
- It needs to be fine-tuned to adapt to domain-specific use cases

GPT - 4 → ChatGPT

Falcon-40B → Falcon-40B-Instruct

Llama-2 →  Alpaca

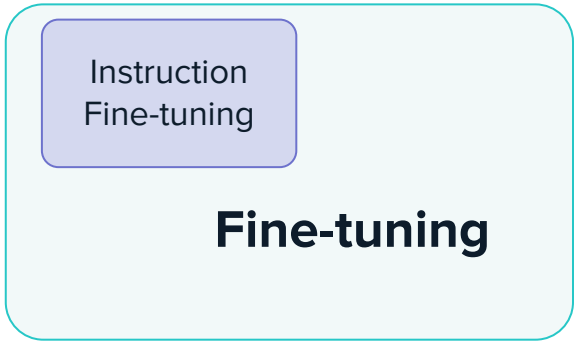
GPT-3 →  Codex/ Copilot

Instruction Fine-tuning

Instruction-tuning

Instruction Fine-tuning is a type of supervised fine-tuning methods.

- It’s also called an “Instruction-following” LLM
- Instruction-tuned models teach LLMs to behave more like a chatbot
- It has better user interface for model interaction and therefore helped increase AI adoption.

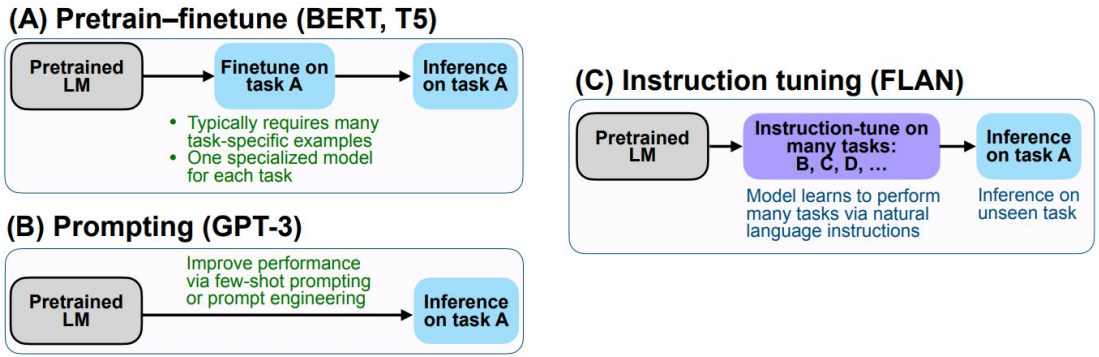


Instruction Fine-tuning

Instruction-tuning

To make a useful conversational agent, subsequent stages of *fine-tuning* help *align* the model with human needs. An aligned model should be

- helpful
- honest
- harmless



Pretrain-finetuning vs prompting vs instruction tuning (Wei et al., 2022).

Instruction Fine-tuning

Instruction-tuning

The pre-trained model is fine-tuned using pairs of prompts **X** and human-written responses **y**. In this case the prompt is *Describe a poodle* and the response is *A poodle is a type of dog that ...*. The model aims to maximize the probability of the response tokens. In this example and optional *<start>* token denotes the beginning of the response.

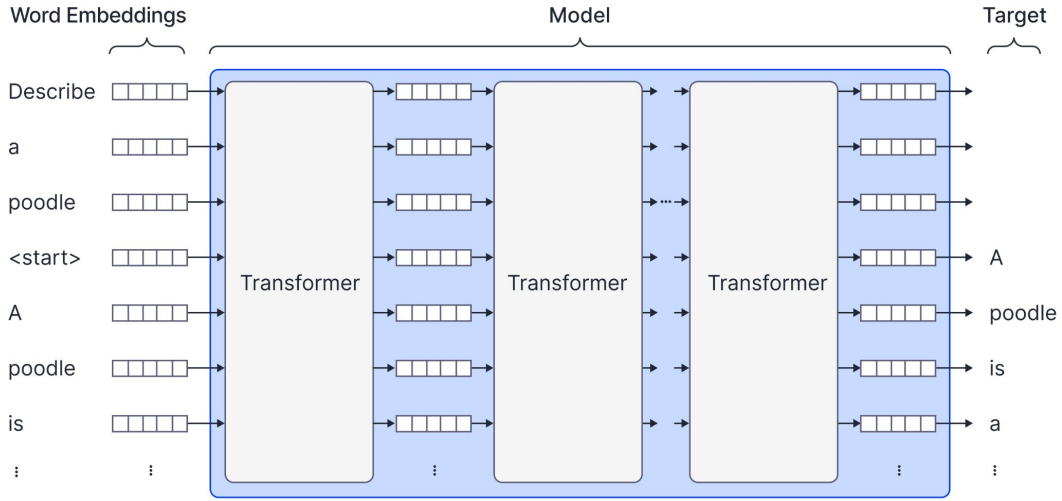


Image source: Training and fine-tuning large language models (blog)

Instruction Fine-tuning Dataset

Instruction-tuning

To teach a pre-trained model to follow instructions, we need to curate and construct very good instruction-following datasets.

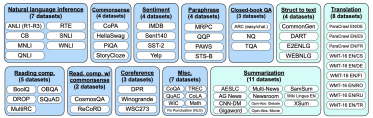
Some of these can be easily pulled from the internet such as:

- Online FAQs
- Customer support conversation history
- Discord chat messages

It can be built from scratch

- For example, collect many training prompts and pay annotators to write responses.

Researchers have also taken existing NLP datasets and reformulating them in question-answer form



Many datasets have been generated/augmented using private/OSS LLMs

Reference; Finetuned Language Models Are Zero-Shot Learners (paper)

Data Limitations

Instruction-tuning

Well-known Instruction-tuning Datasets

- Alpaca data (Taori et al., March 2023)
- Vicuna ShareGPT data
- Databricks-dolly-15k

Data source

- Datasets generated using ChatGPT may inherit biases of the source model or may be noisy.

Data quality

- In most cases, filtering is based on simple heuristics or a pre-trained model, which can result in noisy data.

Cost

- Human-annotated examples are more high quality but are more expensive to obtain.

Domain and language coverage

- Most datasets cover general QA-style use cases and are in English. However, similar methods can be used to obtain data in other domains or languages.

Number of dialog turns

- Most datasets are single-turn, i.e., they consist of a prompt and a single response. Multi-turn data may be necessary to train a more conversational model.

License terms

- Data generated using OpenAI models is subject to the OpenAI terms of use, which prohibit using the data to develop competing models. So look for data with a more permissive license to avoid any legal complications.

Reference: Instruction Tuning Vol. 2 (blog)

Practical Approach to fine-tuning

Instruction-tuning

- Figure out your task
- Collect data related to the task's inputs/outputs
- Generate data if you don't have enough data
- Finetune a small model (e.g. 400M-1B)
- Vary the amount of data you give the model
- Evaluate your LLM to know what's going well vs. not
- Collect more data to improve
- Increase task complexity
- Increase model size for performance

LLM Fine-tuning

RLHF Alignment

Building and Tuning LLMs

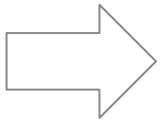
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

Fine-tuning

Parameter Efficient Fine-tuning

Agenda.

Disadvantages of Instruction Fine-tuning

RLHF

Despite the effectiveness, instruction fine-tuning has some disadvantages.

- First and most importantly, collecting this type of data is extremely expensive. Much painstaking work from educated labelers is needed to produce desirable responses for each prompt.
- Second, there are many cases where the correct response is not well-defined. For example, if the prompt is *Tell me a story about a monkey*, no single example response can encompass all the possible responses.
- Third, while it can encourage certain types of responses, there is no way to discourage possible responses, such as those that use discriminatory language or are insufficiently detailed.

Reinforcement Learning from Human Feedback

RLHF

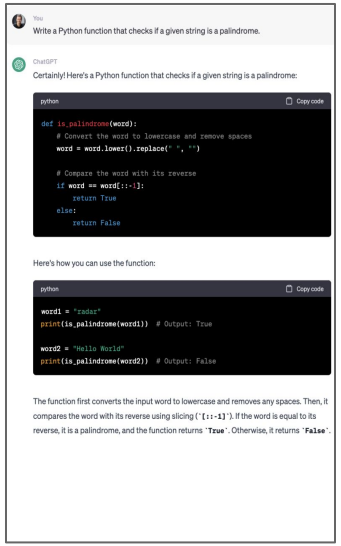
One way to address these issues is to train a model based on ratings of real model responses. The work required to rate an existing response is considerably less than providing that response manually. In addition, this scheme allows for multiple possible valid responses and can be used to actively discourage responses that are harmful. The *reinforcement learning from human feedback* or ***RLHF*** pipeline is used to train language models by encouraging them to produce highly rated responses.

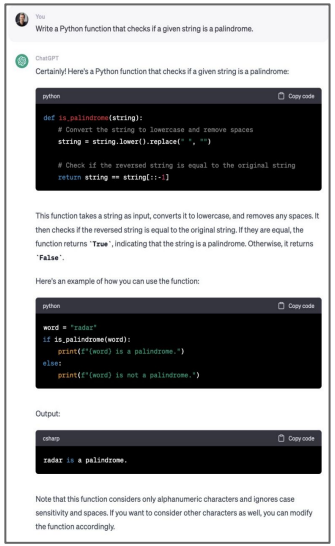
RLHF pipeline proceeds in two stages.

1. First, it trains a reward model using the ranking of responses.
2. Second, the language model is trained using the reward model, which provides an absolute measure of response quality.

Reward Model

RLHF



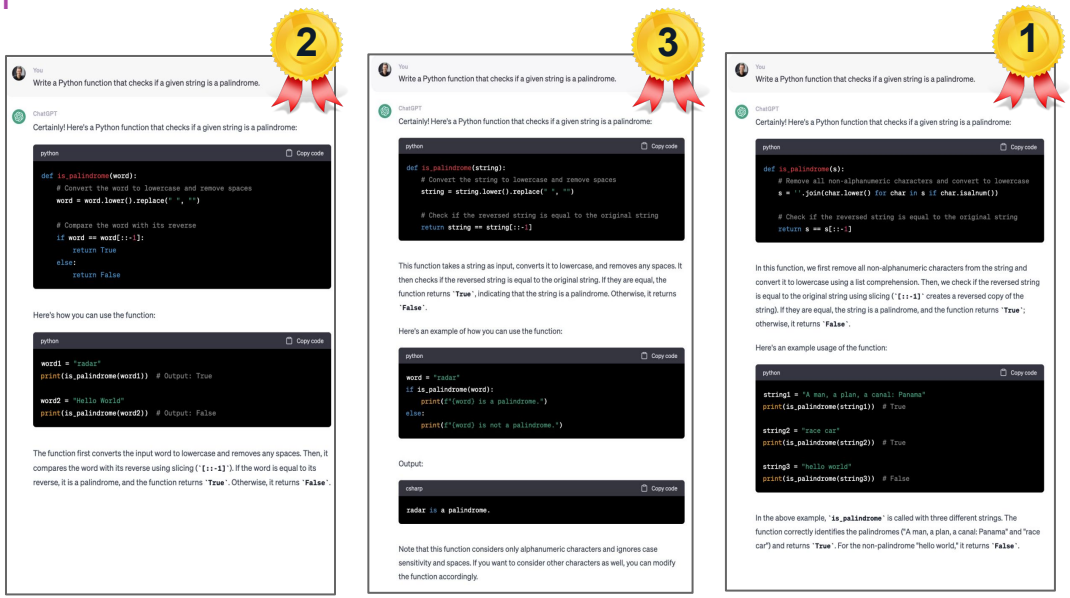


It is difficult to attach an absolute rating to a given response. However, it's easy to compare two or more possible responses and rank which is better.

Reference: State of GPT by Andrej Karpathy (presentation)

Reward Model

RLHF

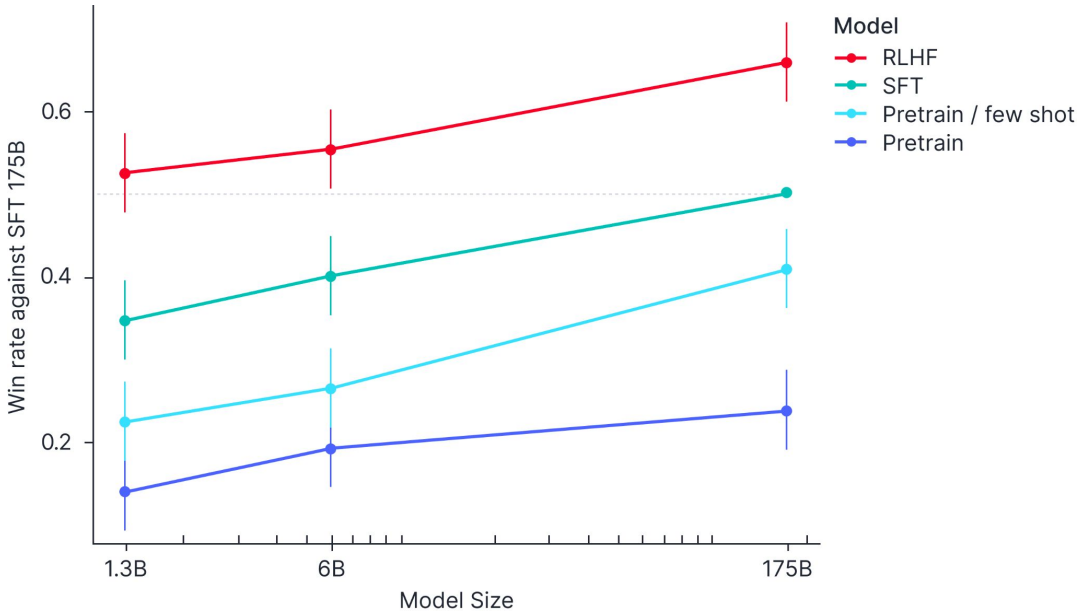


It is difficult to attach an absolute rating to a given response. However, it's easy to compare two or more possible responses and rank which is better.

Reference: State of GPT by Andrej Karpathy (presentation)

RLHF is effective!

RLHF



Reference: State of GPT by Andrej Karpathy (presentation)

LLM Fine-tuning

In-Context Learning

Building and Tuning LLMs

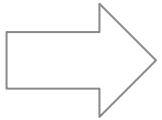
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

Fine-tuning

Parameter Efficient Fine-tuning

Agenda.

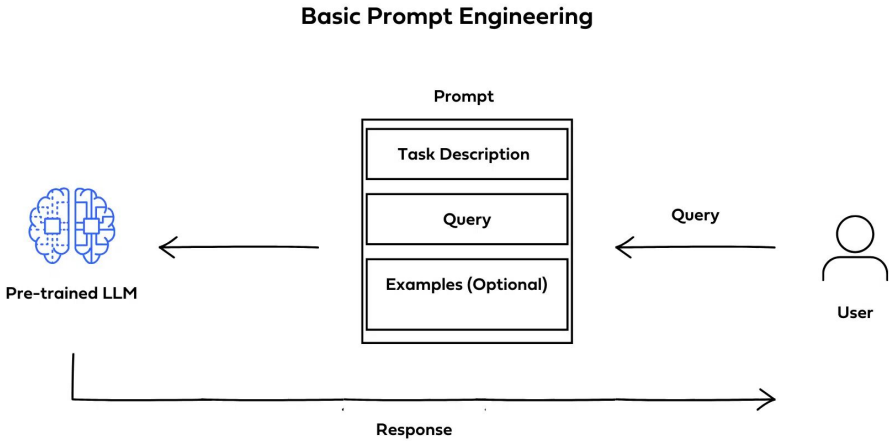
What is In-Context Learning?

In-Context Learning

In-context learning (ICL) is a specific method of prompt engineering where demonstrations of the task are provided to the model as part of the prompt (in natural language).

With ICL, you can use off-the-shelf large language models (LLMs) to solve novel tasks without the need for fine-tuning.

- Used at inference time
- No fine-tuning required
- Performs well with just a few shots of examples



deci

Reference: Full Fine-Tuning, PEFT, Prompt Engineering, and RAG: Which One Is Right for You? (blog)

In-Context Learning

In-Context Learning:

GPT-2 (Radford et al.) and GPT-3 (Brown et al.) are generative large language models (LLMs) pretrained on a general text corpus.

- These models are zero/few-shot learners, i.e., we can directly provide a few examples of a target task via the input prompt.
- This is particularly useful if we access the model only through API.

```
Translate the following German sentences into English:

Example 1:
German: "Ich liebe Eis."
English: "I love ice cream."

Example 2:
German: "Draußen ist es stürmisch und regnerisch"
English: "It's stormy and rainy outside."

Translate this sentence:
German: "Wo ist die naechste Supermarkt?"
```

An example of in-context learning.

Zero-shot vs. One-shot vs. Few-shot

In-Context Learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  cheese =>                           ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← example
3  cheese =>                           ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← examples
3  peppermint => menthe poivrée        ←
4  plush girafe => girafe peluche      ←
5  cheese =>                           ← prompt
```

Reference: Language Models are Few-Shot Learners by OpenAI (paper)

ICL vs Fine-tuning

In-Context Learning

One of the main differences between In-Context Learning and Fine-tuning is that SFT requires more prompt/answer pairs to train and update the models parameters via gradient updates. ICL on the other hand, requires only few examples at the inference time and no model update is required.

Reference: Language Models are Few-Shot Learners by OpenAI (paper)

Effectiveness of ICL

In-Context Learning

Empirical experience shows that ICL can be very effective.

- ICL can achieve SOTA performance and few-shot learning does well as the model becomes better

Language Models are Few-Shot Learners

Tom B. Brown* Benjamin Mann* Nick Ryder* Melanie Subbiah*

Jared Kaplan† Prafulla Dhariwal Arvind Neelakantan Pranav Shyam Girish Sastry

Amanda Askell Sandhini Agarwal Ariel Herbert-Voss Gretchen Krueger Tom Henighan

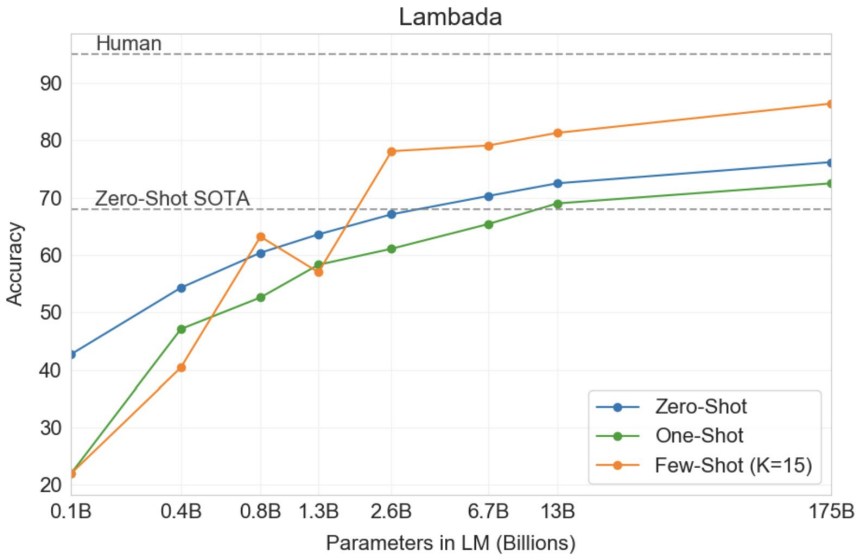
Rewon Child Aditya Ramesh Daniel M. Ziegler Jeffrey Wu Clemens Winter

Christopher Hesse Mark Chen Eric Sigler Mateusz Litwin Scott Gray

Benjamin Chess Jack Clark Christopher Berner

Sam McCandlish Alec Radford Ilya Sutskever Dario Amodei

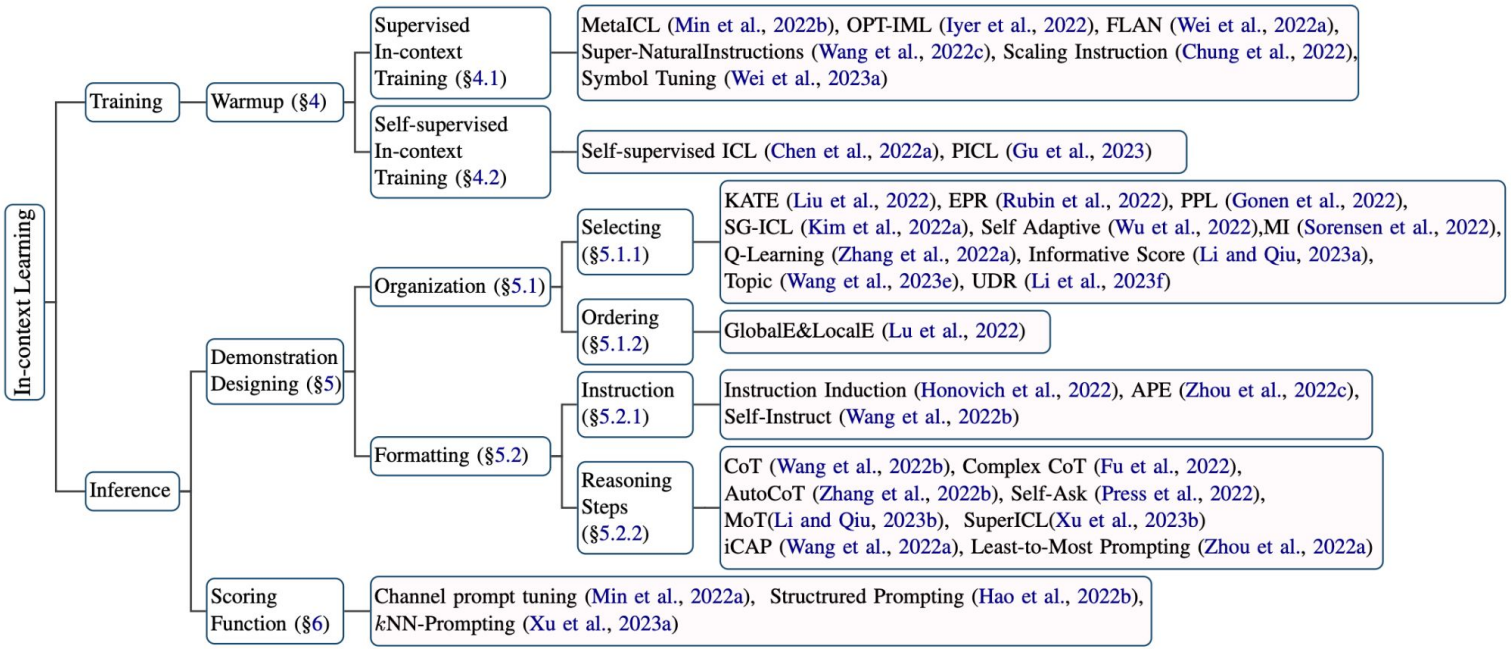
OpenAI



Reference: Language Models are Few-Shot Learners by OpenAI (paper)

Effectiveness of ICL

In-Context Learning



Reference: A Survey on In-context Learning (paper)

Few-shot Prompt Template

LangChain

In practice, we can use higher-level APIs such as LangChain to manage the prompt templates for ICL.

```
from langchain.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate.from_messages([
    ("human", "What is the capital of {country}?"),
    ("ai", "The capital of {country} is {capital}.")
])

messages = chat_template.format_messages(
    country="Canada",
    capital="Winnipeg"
)

print(messages)
```

Reference:

- Few-shot prompt templates (LangChain)
- Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

For best results, we generally recommend using the latest, most capable models.

Reference: Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

Less effective ❌:

```
Summarize the text below as a bullet point list of the most important points.

{text input here}
```

Better ✅:

```
Summarize the text below as a bullet point list of the most important points.

Text: """
{text input here}
"""
```

Reference: Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

Less effective ❌:

```
Write a poem about OpenAI.
```

Better ✅:

```
Write a short inspiring poem about OpenAI, focusing on the recent DALL-E product
```

Reference: Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

Less effective ❌:

```
Extract the entities mentioned in the text below. Extract the following 4 entity

Text: {text}
```

Show, and tell - the models respond better when shown specific format requirements. This also makes it easier to programmatically parse out multiple outputs reliably.

Better ✅:

```
Extract the important entities mentioned in the text below. First extract all co

Desired format:
Company names: <comma_separated_list_of_company_names>
People names: -||-
Specific topics: -||-
General themes: -||-

Text: {text}
```

Reference: Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

Less effective ❌:

```
The description for this product should be fairly short, a few sentences only, a
```

Better ✅:

```
Use a 3 to 5 sentence paragraph to describe this product.
```

Reference: Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

Less effective ❌:

```
The following is a conversation between an Agent and a Customer. DO NOT ASK USER

Customer: I can’t log in to my account.
Agent:
```

Better ✅:

```
The following is a conversation between an Agent and a Customer. The agent will

Customer: I can’t log in to my account.
Agent:
```

Reference: Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

Less effective ❌:

```
# Write a simple python function that
# 1. Ask me for a number in mile
# 2. It converts miles to kilometers
```

In this code example below, adding “*import*” hints to the model that it should start writing in Python. (Similarly “SELECT” is a good hint for the start of a SQL statement.)

Better ✅:

```
# Write a simple python function that
# 1. Ask me for a number in mile
# 2. It converts miles to kilometers

import
```

Reference: Best practices for prompt engineering with openai api (article)

Prompting Best Practices (OpenAI)

LangChain

1. Use the latest model
2. Put instructions at the beginning of the prompt and use ### or """ to separate the instruction and context
3. Be specific, descriptive and as detailed as possible about the desired context, outcome, length, format, style, etc
4. Articulate the desired output format through examples
5. Reduce “fluffy” and imprecise descriptions
6. Instead of just saying what not to do, say what to do instead
7. Code Generation Specific - Use “leading words” to nudge the model toward a particular pattern
8. Start with zero-shot, then few-shot (example), neither of them worked, then fine-tune

✅ Zero-shot

```
Extract keywords from the below text.

Text: {text}

Keywords:
```

✅ Few-shot - provide a couple of examples

```
Extract keywords from the corresponding texts below.

Text 1: Stripe provides APIs that web developers can use to integrate payment pr
Keywords 1: Stripe, payment processing, APIs, web developers, websites, mobile a
##
Text 2: OpenAI has trained cutting-edge language models that are very good at un
Keywords 2: OpenAI, language models, text processing, API.
##
Text 3: {text}
Keywords 3:
```

Reference: Best practices for prompt engineering with openai api (article)

LLM Fine-tuning

Indexing

Building and Tuning LLMs

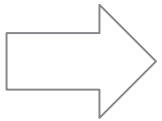
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

Fine-tuning

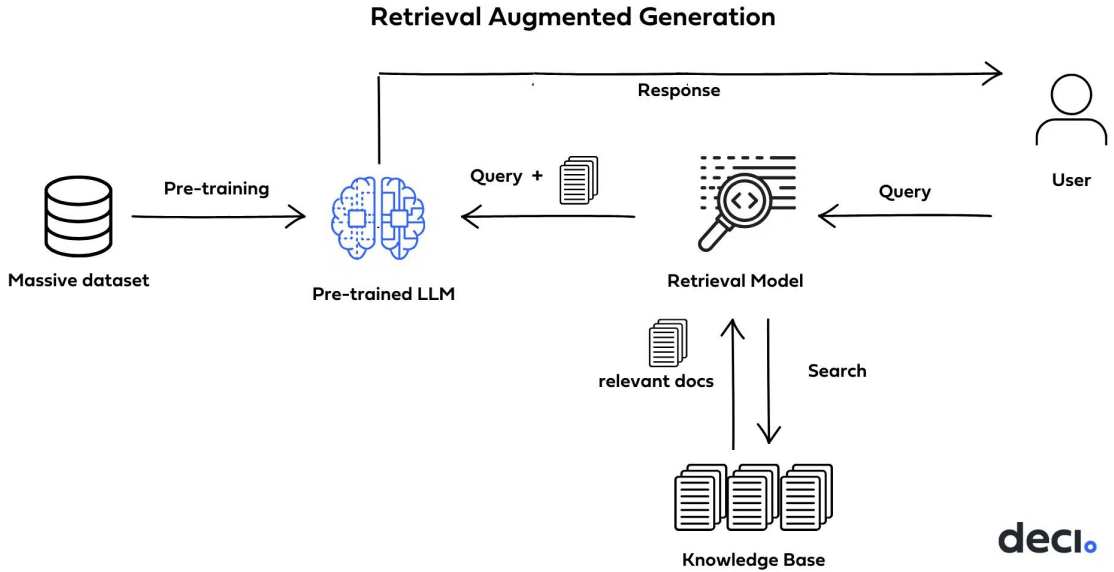
Parameter Efficient Fine-tuning

Agenda.

Retrieval Augmented Generation

Indexing

RAG (Retrieval Augmented Generation) is very popular and has wide adoptions. It's an alternative to fine-tuning and has practical use cases. It requires more engineering and devops effort.



Source: Open Source LLMs, Fine-Tunes and RAG Based Vector Store APIs (Abacus AI Blog)

Using the Pre-trained LLM

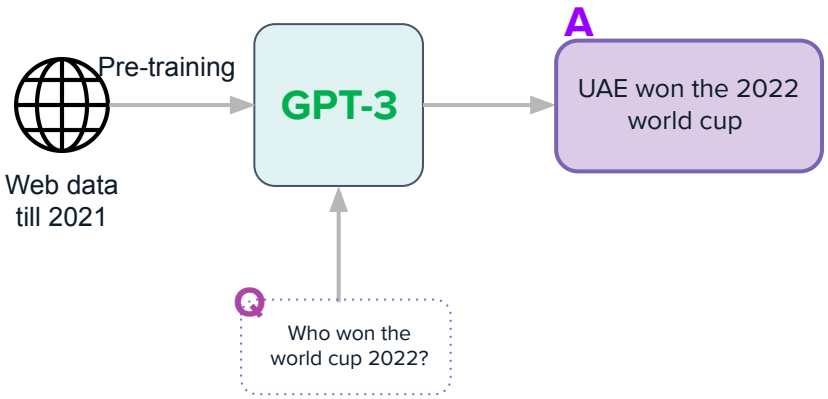
Indexing

Challenge

LLM does a good job on completion but hallucinate a lot. Answers are not factual

Solution

Instruction tune the model and add RLHF



Using the Fine-tuned LLM

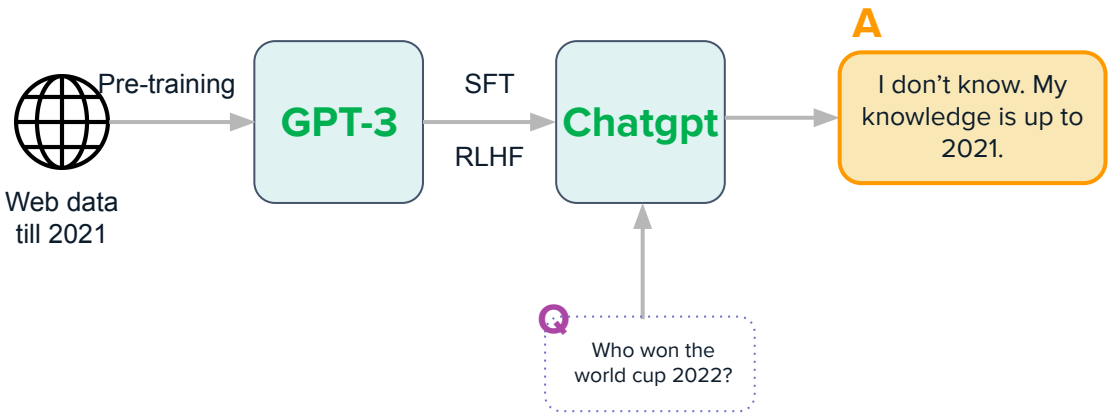
Indexing

Challenge

Follows instruction, but knowledge is not up to date.

Solutions

- Re-train the LLM with fresh data
- Fine-tuning
- Retrieval from external knowledge base



Retraining the LLM (new knowledge)

Indexing

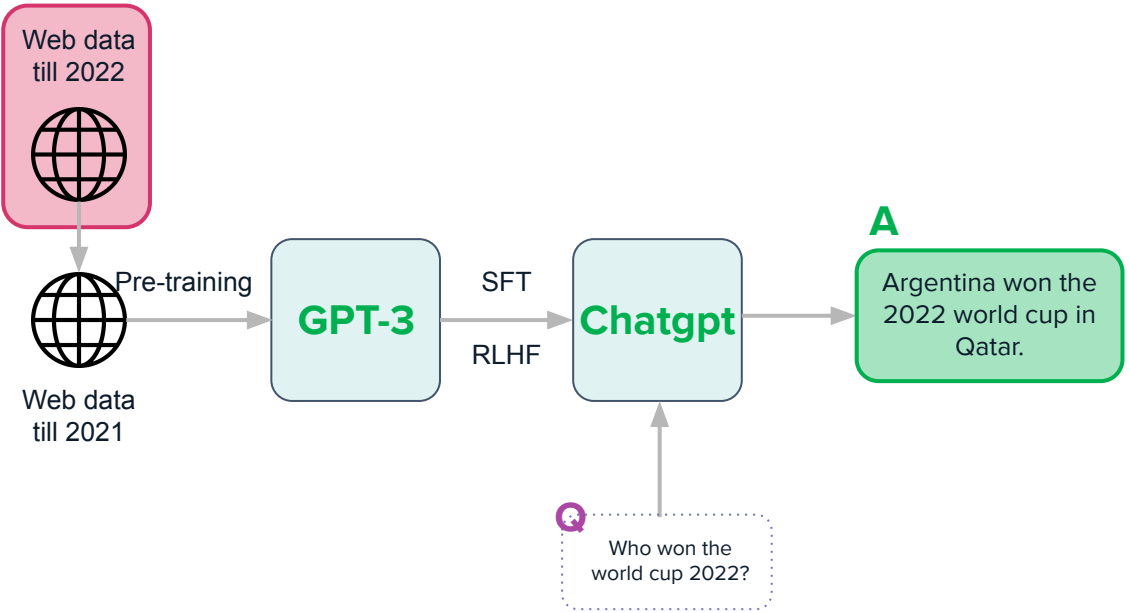
Challenge

Follows instruction, but knowledge is not up to date.

Solution #1 (Retraining)

Re-train the LLM and bring the knowledge up to date

- Too expensive
- Knowledge is always lagging



Retraining the LLM (new knowledge)

Indexing

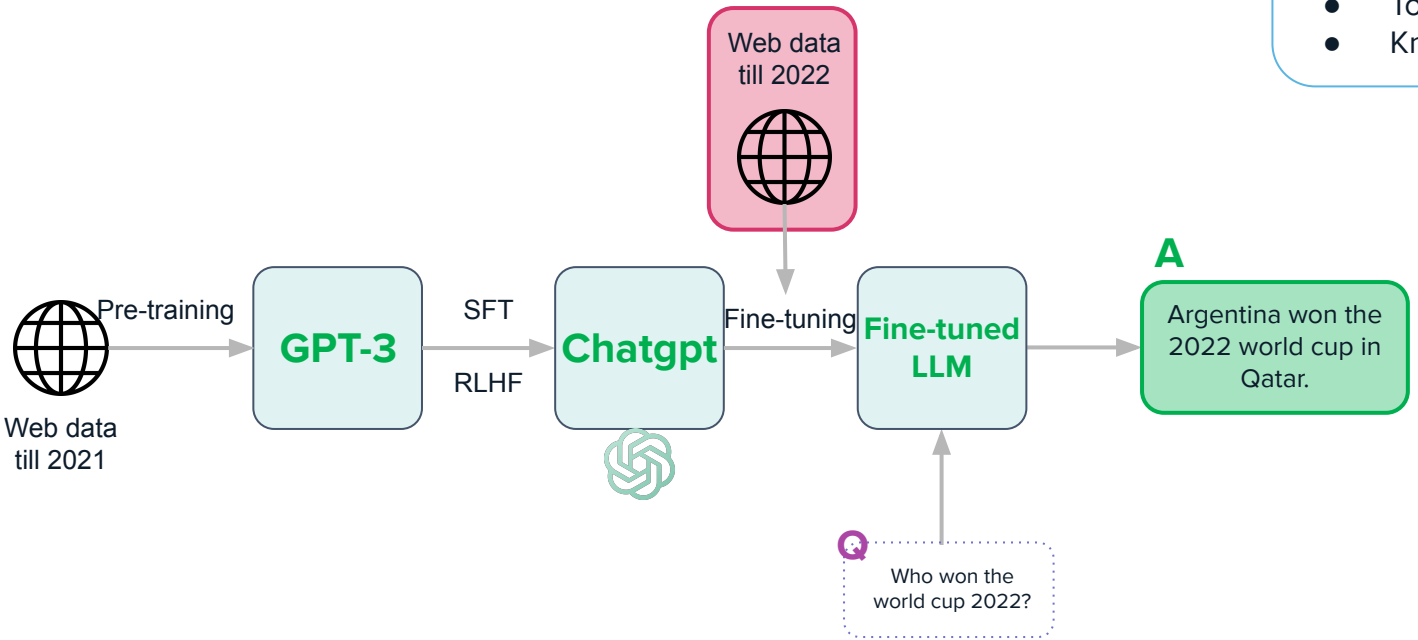
Challenge

Follows instruction, but knowledge is not up to date.

Solution #2 (Fine-tuning)

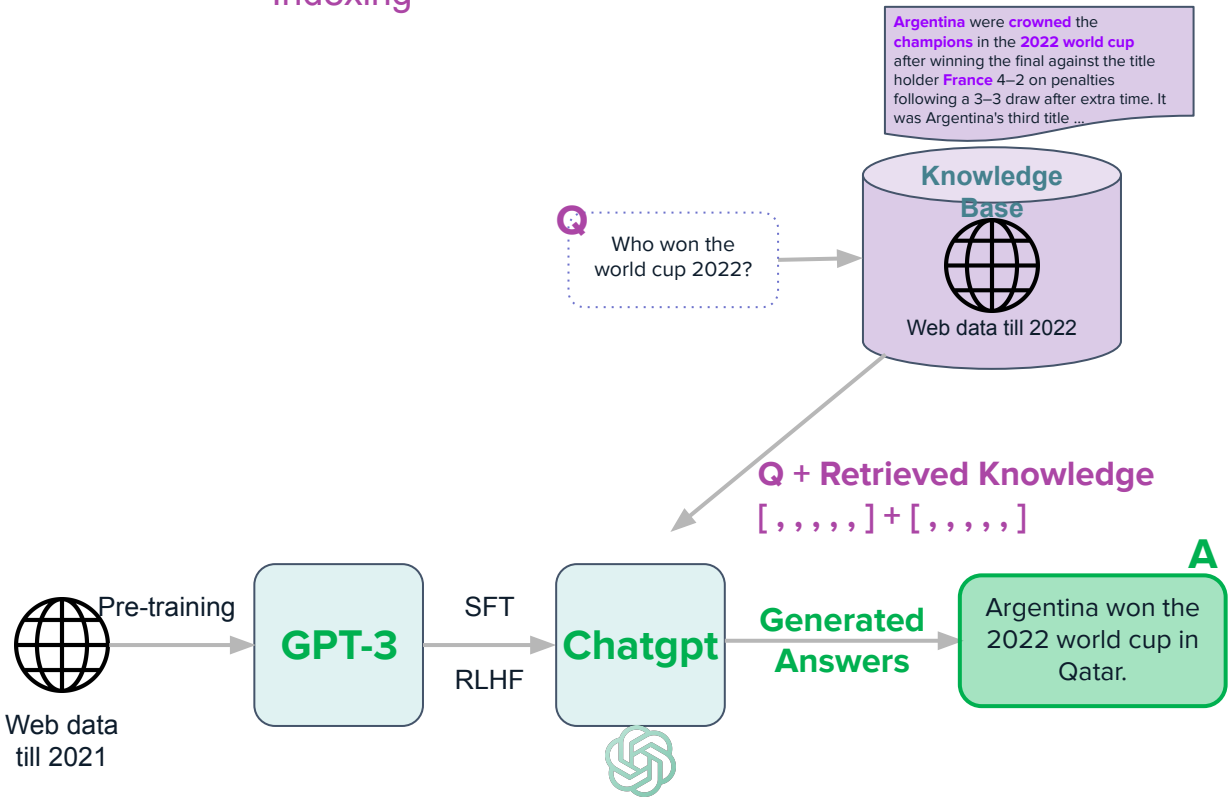
Re-train the LLM and bring the knowledge up to date

- Too expensive
- Knowledge is always lagging



Retrieving from External Knowledge

Indexing



Challenge

Follows instruction, but knowledge is not up to date.

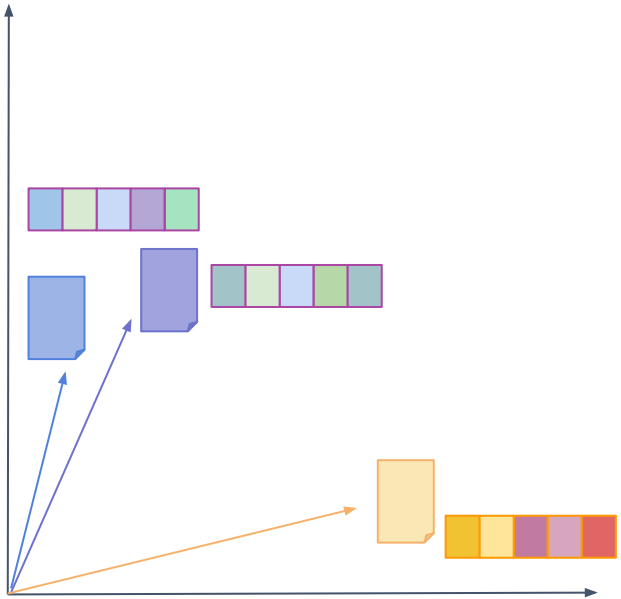
Solution #3 (RAG)

Retrieval Augmented Generation

- Retrieve from external knowledge base first
- Send query + retrieved knowledge to LLM
- LLM generates the answer

Vector Similarity

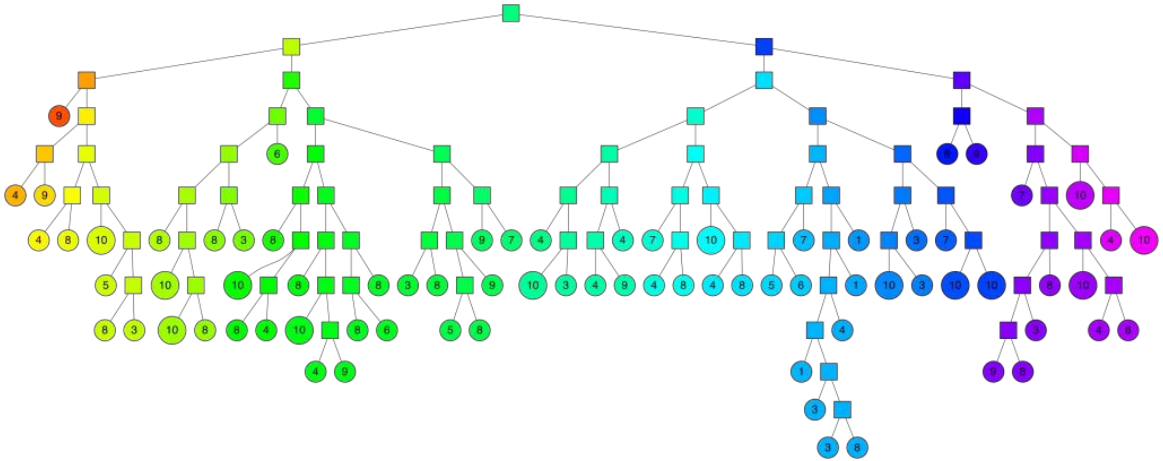
Indexing

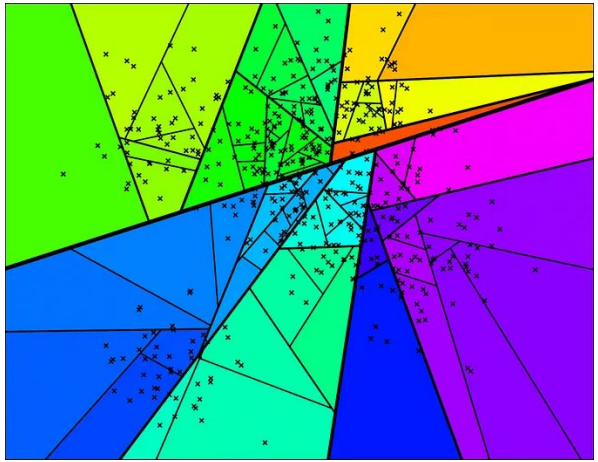


Vector Similarity

Indexing

What is

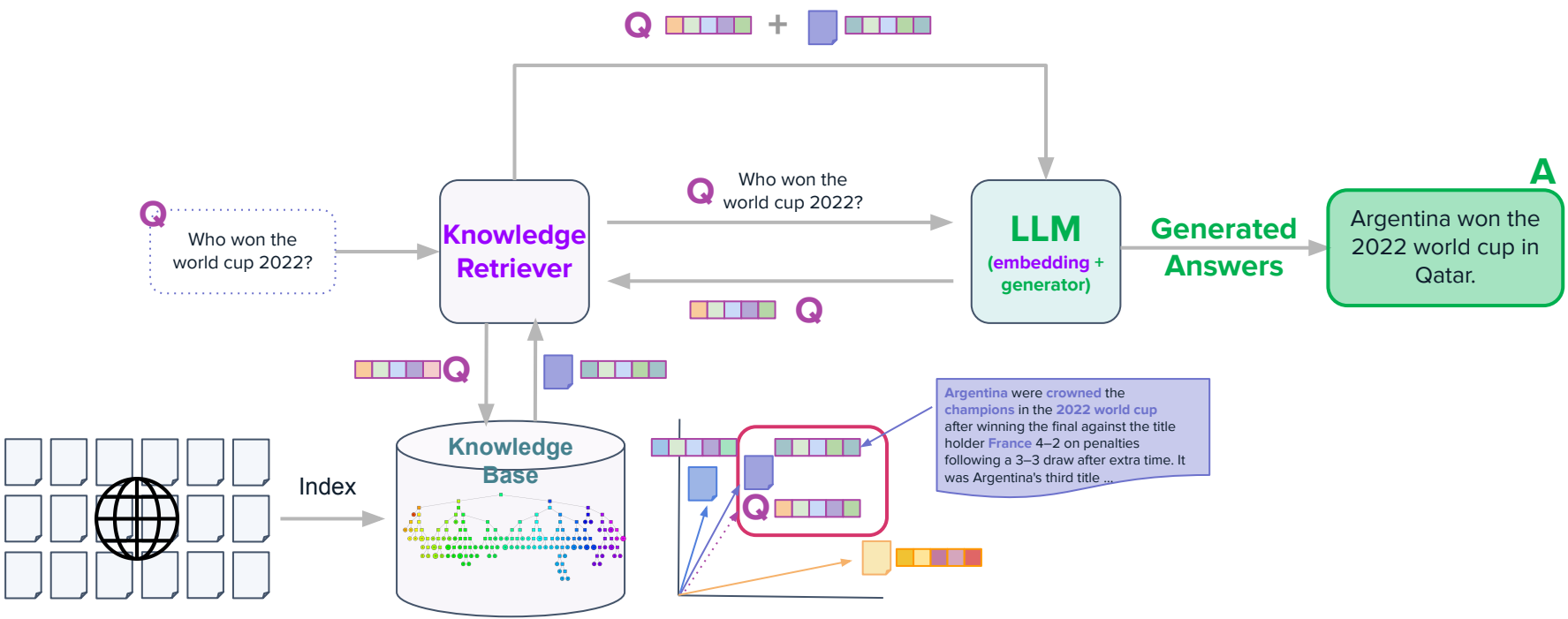




Reference: Nearest neighbor methods and vector models (part1 part2)

RAG Workflow

Indexing



LLM Fine-tuning

Fine-tuning

Building and Tuning LLMs

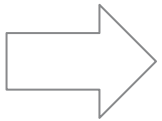
Introduction

Pre-training

Fine-tuning Intro

Instruction Tuning

Alignment



In-Context Learning

Indexing

Fine-tuning

Parameter Efficient Fine-tuning

Agenda.

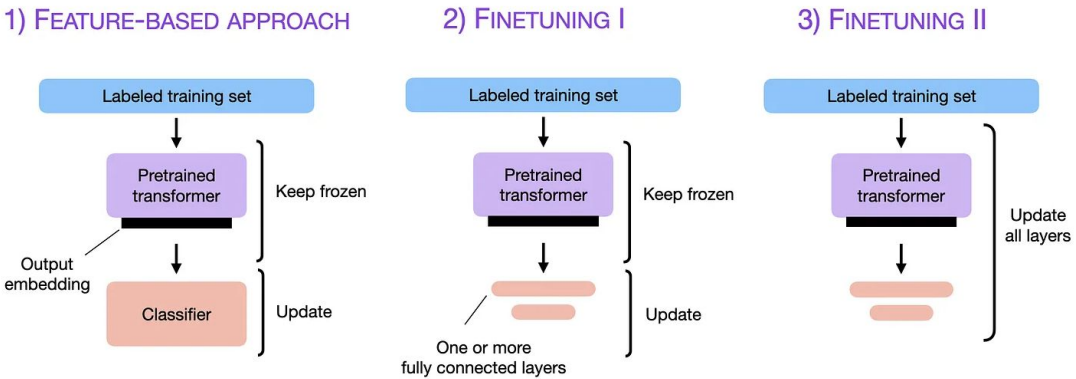
LLM Fine-tuning

Fine-tuning

While prompting and instruction-tuning can yield the desired results, the subsequent step to enhance performance is to proceed with fine-tuning.

Fine-Tuning:

- We take a pre-trained a Large Language Model and finetune it on a target task.



The 3 conventional feature-based and finetuning approaches.

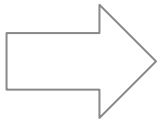
https://magazine.sebastianraschka.com/p/finetuning-large-language-models

LLM Fine-tuning

PEFT

Building and Tuning LLMs

Introduction
Pre-training
Fine-tuning Intro
Instruction Tuning
Alignment



In-Context Learning
Indexing
Fine-tuning
Parameter Efficient Fine-tuning

Agenda.

Motivation for Parameter-Efficient Fine-Tuning

PEFT Introduction

In the previous session, we learned that fine-tunings can bring behavior changes to the LLMs and help gain new knowledge and reduce hallucinations.

However, fully tuning a LLM is very costly.

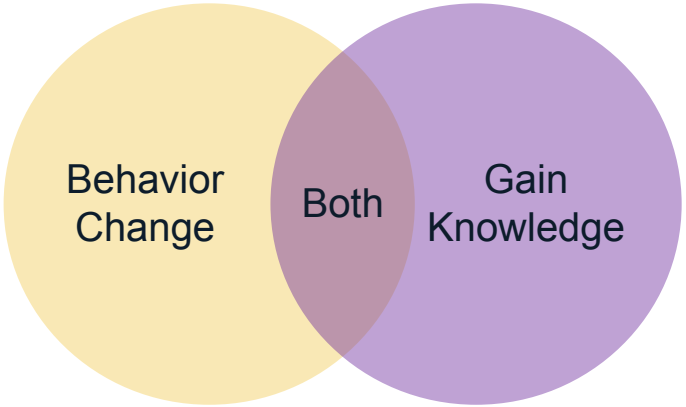


Image adapted from Lamini's deeplearning.ai fine-tuning lecture

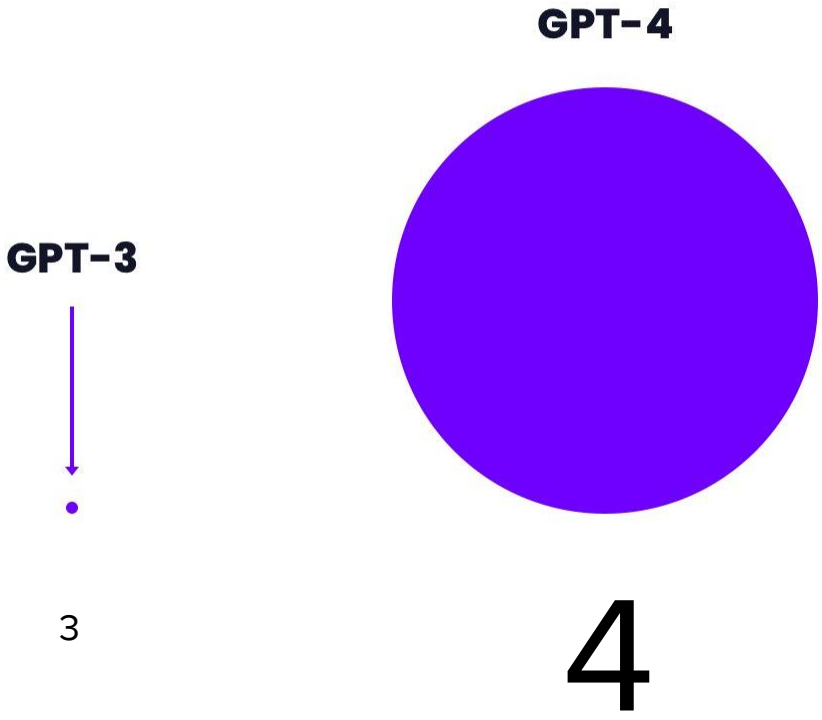
Motivation for Parameter-Efficient Fine-Tuning

PEFT Introduction

LLM model size has been growing rapidly

- Publically available model sizes: 350M ➜ 176B
- Single-GPU RAM: 16Gb ➜ 80Gb

- Model size scales almost two orders of magnitude quicker than single-GPU memory



Motivation for Parameter-Efficient Fine-Tuning

PEFT Introduction

Low-Rank Adaptation, which is one of the most effective adapter-based fine-tuning method, has proven to achieve on-par or even better performance on certain performance valuation metrics compared to full-parameter fine-tuning of GPT-3 175B.

- It only requires only 37.7 million parameters, much less than the full-parameter fine-tuning approach.

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($Adapter^H$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($Adapter^H$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Performance of different adaptation methods on GPT-3 175B. LoRA performs better than prior approaches, including full fine-tuning, with much less trainable parameters.

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

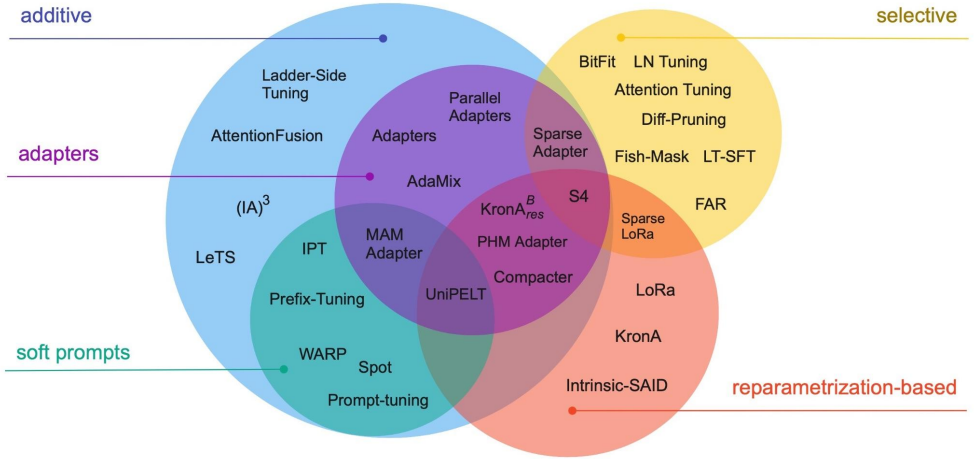
Reference: LoRA: Low-Rank Adaptation of Large Language Models (paper)

A Taxonomy of Fine-tuning Methods

PEFT Introduction

The PEFT landscape can be quite overwhelming. Methods can be classified in multiple ways.

- They may be differentiated by their underlying approach or conceptual framework
  - does the method introduce new parameters to the model, or
  - does it fine-tune a small subset of existing parameters?
- Alternatively, they may be categorized according to their primary objective:
  - does the method aim to minimize memory footprint,
  - or only storage efficiency?



Reference: Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning (paper)

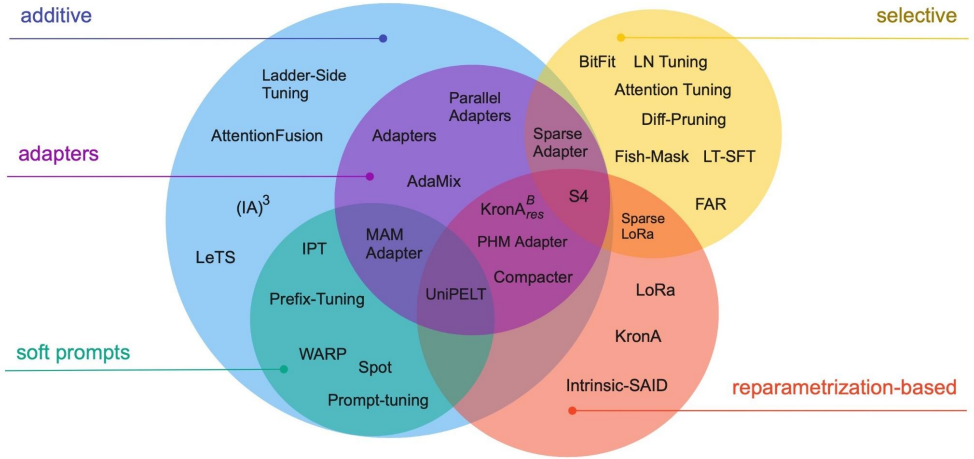
A Taxonomy of Fine-tuning Methods

PEFT Introduction

In the Scaling Down to Scale Up paper, the authors created a taxonomy that categorizes PEFT into the following categories

- Additive methods
- Selective methods
- Reparametrization-based methods
- Hybrid methods

In this course, we will explore the adapters and reparametrization-based methods such as LoRA and QLoRA.



Reference: Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning (paper)

LoRA: Low-Rank Adaptation of LLMs

PEFT Introduction

- LoRA
  - Fewer trainable parameters: 10000x less for GPT3
  - Less GPU memory: 3x less for GPT3
  - On-par or lightly below accuracy to fine-tuning
  - Some inference latency
- Train new weights in some layers, freeze main weights
  - New weights: rank decomposition matrices of original weights' change
  - At inference, merge with main weights

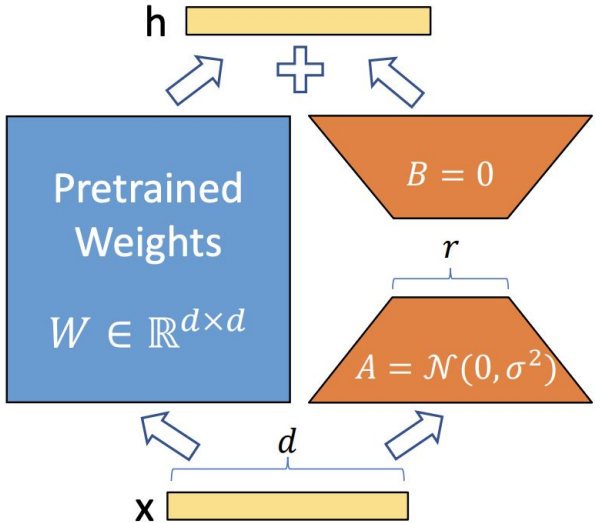


Image Source: LoRA: Low-Rank Adaptation of Large Language Models (paper)